

编译原理实验二实验报告

已实现功能

本实验在词法分析和语法分析的基础上，实现了 C-语言的语义分析与类型检查功能。根据实验一，程序首先由 Flex 和 Bison 完成输入源程序的词法、语法分析，并构建程序语法树；当词法和语法阶段没有错误后，再调用语义分析模块，对语法树进行递归遍历，完成符号表维护、类型构造、表达式类型推导和语义错误检查。

本程序除了实现了实验要求中的 1 至 17 类基础语义检查，还另外实现了三个选做方向。第一，支持函数声明，能检查函数声明后未定义的错误，以及多次声明、声明与定义之间返回类型或参数列表不一致的错误，即错误类型 18 和 19；第二，支持可嵌套作用于，允许内层语句块定义与外层同名的局部变量，并在离开语句块后恢复外层变量的可见性；第三，支持结构体的结构等价判断，即在结构体赋值和类型比对时，根据字段顺序和字段类型来判断两个结构体类型是否等价。

编译与运行方法

本程序使用 Makefile 进行编译。进入 code 目录后，执行：

```
make
```

即可生成可执行文件 cc。如果需要重新编译，可以按顺序执行：

```
make clean
```

```
make
```

程序运行方式为：

```
./cc <input_file>
```

例如: `./cc test/test_01_undefined_var.cmm`

如果程序存在语义错误,则会根据实验要求在标准输出中打印错误类型、行号和说明问题。

为了便于测试,我将所有测试用例放在 `test/` 目录下,其中基础测试用例命名为: `test_01_xxx.cmm`,共 17 个。选做测试用例命名为 `test_optional_01_xxx.cmm`,共 6 个。此外,我还编写了 `run_tests.sh` 测试脚本一次运行所有测试。

实现亮点与运行方式

本程序的核心实现集中于 `semantic.c` 中。相比于直接在语法分析中插入大量语义动作,本程序将语义分析独立为一个模块,在语法树构建之后统一遍历处理。这样使词法、语法、语义三部分边界清晰,后续调试时也能根据错误类型直接定位到变量表、函数表、类型系统或表达式分析的对应逻辑。

程序的第一个主要设计亮点是统一的类型表示。`semantic.c` 中使用同一套 `Type` 结构表示基本类型、数组类型、结构体类型和错误类型。数组类型通过递归方式表示多维数组,结构体类型通过字段链表表示成员信息;同一个字段链表结构还被复用到函数形参和实参列表。这样处理可以让数组访问、结构体赋值、函数调用的参数匹配检查等都可以统一到类型构造和类型比较问题,减少了重复判断逻辑。

第二个亮点是支持嵌套作用域的变量符号表。变量表采用“哈希表 + 作用域链表”的结构:哈希表负责快速查找变量,作用域链表负责记录当前层级中新定义的变量。进入语句块时创建新的作用域,退出语句块时只删除本层变量;由于内层变量插入在哈希桶前部,查找时会优先命中最近作用域中的定义。这个设计能令程序可以正确处理全局变量、函数形参、局部变量以及内层语句块变量之间的遮蔽关系。

第三个亮点是错误恢复机制。程序引入 `TYPE_ERROR` 作为错误哨兵类型,当某个子表达式已经因为未定义变量、未定义函数、非法字段访问等原因报错时,后续父表达式能接收到错误类型,并尽量避免继续产生由一个根因引起的连锁报错。这样既能继续遍历整颗语法树、发现多个独立错误,又能让输出更集中地反映真正的错误来源。

整体而言,本程序的实现重点在于将语义分析组织成几个相对独立的部分:类型系统、变量符号表、函数表、结构体表和表达式分析。各部分之间通过统一的类型和符号查询结构完成语义检查,因此代码具有较好的可维护性,也方便后续继续扩展中间代码生成等编译阶段。